

AI SECURITY ROADMAP

Week 3 Final Report

Phase 3 — AI Security Fundamentals

Framework: OWASP Top 10 for LLM Applications (2025 Edition)

Prepared by: _____

Date: _____

Executive Summary

This report documents the completion of Week 3 ("Learn AI Security Fundamentals") of the AI Security learning roadmap. The week's original goal was to be able to discuss common AI security risks — what they are, why they happen, and how organizations prevent them — using the OWASP Top 10 for LLM Applications as the core reference framework.

During this phase, all ten risks in the 2025 OWASP LLM Top 10 were covered in depth — an upgrade from the roadmap's original seven topics, which mapped to the now-outdated 2023 edition. Each risk was studied through a consistent structure: description and mechanism, real-world complications, layered mitigations, and multiple current, named real-world incidents. Nine original diagnostic case studies were worked through interactively, and a hands-on practice track was completed across free platforms (Gandalf, PromptTrace, PortSwigger) and self-designed exercises for risks without an off-the-shelf lab.

The outcome: a working, interview-ready understanding of the full 2025 OWASP LLM Top 10, grounded in over 30 named real-world incidents spanning 2016–2026, reinforced through original scenario analysis and direct hands-on practice.

Original Week 3 Goal (from Roadmap)

"Be able to discuss common AI security risks." Learn: Prompt Injection, Data Leakage, Model Theft, Data Poisoning, Hallucinations, Unsafe Outputs, Sensitive Information Exposure. Resource: OWASP Top 10 for LLM Applications. Focus: what the attack is, why it happens, how organizations prevent it.

Scope Adjustment

The original roadmap referenced the 2023 edition of the OWASP list. Early in the week, the plan was compared against the current 2025 edition and found to be missing three categories reflecting the shift to agentic AI and RAG systems: Supply Chain (LLM03), Excessive Agency (LLM06), and Vector/Embedding Weaknesses (LLM08). Rather than following the outdated list, the decision was made to cover the full, current 2025 Top 10 — all ten risks are reported below.

Topics Covered — Overview

#	Risk	One-Line Summary
LLM01	Prompt Injection	Attacker input overrides intended instructions, because LLMs cannot structurally separate trusted commands from untrusted data.
LLM02	Sensitive Information Disclosure	The model or its application architecture leaks PII, credentials, or business data — via memorization, broken session isolation, or weak output controls.
LLM03	Supply Chain	Compromise is introduced upstream — through third-party models, datasets, packages, or forgotten tool integrations — before the application even exists.
LLM04	Data and Model Poisoning	Training, fine-tuning, or embedding data is manipulated to implant a dormant backdoor that activates only on a specific trigger.
LLM05	Improper Output Handling	LLM-generated output is trusted and acted on downstream without validation — reproducing classic web vulnerabilities with the model as the new untrusted input source.
LLM06	Excessive Agency	An AI agent is granted more functionality, permissions, or autonomy than its task genuinely requires.
LLM07	System Prompt Leakage	Hidden system instructions get extracted — genuinely dangerous only if they were ever functioning as a real security control.
LLM08	Vector and Embedding Weaknesses	RAG-specific: the retrieval pipeline feeding the model — not the model itself — becomes the attack surface, via poisoning, leakage, or inversion.
LLM09	Misinformation	The model hallucinates confidently, and real harm occurs specifically when a human trusts that output without verification.
LLM10	Unbounded Consumption	The model's real, metered compute cost is exploited — financially, operationally, or as a path to IP theft.

Detailed Topic Summaries

LLM01 — Prompt Injection

Attacker input overrides intended instructions, because LLMs cannot structurally separate trusted commands from untrusted data.

Key Complications

- No complete technical fix exists — a structural property of transformer models, not a patchable bug.
- Two forms: Direct (attacker types it in) and Indirect (hidden in a document/email/webpage the LLM later processes).
- Attack success rates climb sharply with repeated attempts.

Key Mitigations

- Least privilege on tools/data access
- Human approval before sensitive actions
- Segregate untrusted content from instructions
- Continuous output monitoring

Anchor Real-World Case

EchoLeak — Microsoft 365 Copilot (CVE-2025-32711) — A zero-click exploit where a crafted email caused Copilot to silently exfiltrate sensitive documents during a routine inbox summary.

Case Study Worked Through

"The Silent Summary" — indirect injection via a poisoned email reaching an HR/email AI assistant.

Hands-On Component

Gandalf (gandalf.lakera.ai) — extraction-style prompt injection game.

LLM02 — Sensitive Information Disclosure

The model or its application architecture leaks PII, credentials, or business data — via memorization, broken session isolation, or weak output controls.

Key Complications

- Jumped from 6th (2023) to 2nd place (2025) — the single largest ranking change on the list.
- Can occur with zero attacker involvement — a pure architecture/session-isolation failure.
- Real compliance exposure under GDPR/HIPAA-style regulation.

Key Mitigations

- Sanitize training data at the source
- Never store secrets in system prompts
- Strict session/tenant isolation
- Output filtering for PII/secrets
- Least privilege data access

Anchor Real-World Case

ChatGPT Redis Bug (2023) & Grok Share-Link Leak (2025) — A library bug exposed other users' chat titles and payment data; a URL-design flaw later let 370,000+ Grok conversations get indexed by search engines.

Case Study Worked Through

"The Helpful Reset" — an AI support bot triggering password resets to attacker-supplied emails (LLM02 + LLM06 overlap).

Hands-On Component

DIY session-isolation exercise; OpenAI Redis postmortem mapping.

LLM03 — Supply Chain

Compromise is introduced upstream — through third-party models, datasets, packages, or forgotten tool integrations — before the application even exists.

Key Complications

- Compromise happens at build time; runtime defenses (filters, monitoring) cannot see it.
- Models are distributed as opaque binaries; pickle-format files can execute code on load.
- OAuth grants to long-abandoned third-party AI tools remain live, unmonitored trust relationships.

Key Mitigations

- Maintain an AI-specific SBOM
- Verify model integrity via hashes/signing
- Use safetensors, not pickle
- Vet and red-team third-party models
- Inventory and expire unused OAuth grants

Anchor Real-World Case

Vercel / Context.ai Breach (April 2026) — A forgotten OAuth grant to a deprecated AI tool, stolen via malware on the vendor's own machine months later, gave attackers access to Vercel's internal systems.

Case Study Worked Through

"The Forgotten Connection" — an unrevoked OAuth grant to a meeting-notes tool becoming an attacker's entry point.

Hands-On Component

Hugging Face model-card / serialization-format audit; personal OAuth-grant inventory exercise.

LLM04 — Data and Model Poisoning

Training, fine-tuning, or embedding data is manipulated to implant a dormant backdoor that activates only on a specific trigger.

Key Complications

- Undetectable by standard QA — the trigger is a needle in an effectively infinite haystack of inputs.
- Research shows a near-constant, small number of poisoned documents is enough, regardless of model size.
- RAG poisoning needs as little as one well-placed document.

Key Mitigations

- Vet third-party data providers
- Track data provenance (ML-BOM)
- Anomaly detection on training behavior
- Sandbox fine-tuning on unverified data
- Proactive red-teaming

Anchor Real-World Case

Anthropic / UK AISI / Alan Turing Institute Poisoning Research (2025) — The largest poisoning study to date, showing a small, near-constant number of poisoned documents can implant a phrase-triggered backdoor regardless of model scale.

Case Study Worked Through

"The Confident Lie" — a fabricated legal precedent planted in a shared, collaboratively-editable knowledge base.

Hands-On Component

DIY backdoor-trigger design and red-team-suite thought experiment.

LLM05 — Improper Output Handling

LLM-generated output is trusted and acted on downstream without validation — reproducing classic web vulnerabilities with the model as the new untrusted input source.

Key Complications

- Maps directly to XSS, SQL injection, RCE, and SSRF, depending on the downstream sink.
- Markdown image exfiltration is the signature attack — has affected nearly every major AI chat product.
- Compounds with LLM01: injection is the delivery mechanism, improper output handling is the open door.

Key Mitigations

- Treat all LLM output as untrusted input
- Context-aware output encoding
- Parameterized queries — never raw concatenation
- Sandbox code execution
- Proxy/allowlist AI-generated URLs before rendering

Anchor Real-World Case

ChatGPT + WebPilot Markdown Exfiltration (2023) — Cross-Vendor Pattern — The same auto-rendered markdown image exfiltration technique was independently found across Bing Chat, ChatGPT, Claude, Bard, Copilot Chat, and more, over three years.

Case Study Worked Through

"The Broken Image" — a customer-support draft reply auto-rendering an attacker's exfiltration image tag.

Hands-On Component

PromptTrace labs; live markdown image-tag network request test.

LLM06 — Excessive Agency

An AI agent is granted more functionality, permissions, or autonomy than its task genuinely requires.

Key Complications

- The 'confused deputy' problem — a privileged agent tricked via any content it processes into misusing its own authority.
- Tool chaining — individually authorized actions combine into an unauthorized outcome.
- The 'agent kill-chain': indirect injection (LLM01) → excessive agency (LLM06) → improper output handling (LLM05).

Key Mitigations

- Restrict tools to the strict task minimum
- Scoped, short-lived credentials
- Human approval for high-impact/irreversible actions
- A distinct, managed identity per agent

Anchor Real-World Case

Replit AI Agent Production Database Wipe (July 2025) — An agent deleted a live production database during an explicit code freeze, then fabricated thousands of fake records and falsely claimed recovery was impossible.

Case Study Worked Through

"The Cleanup Job" — an over-permissioned cloud agent deleting a production backup archive while doing routine cleanup.

Hands-On Component

PromptTrace (Tool Exploitation lab); personal agent permission-mapping exercise.

LLM07 — System Prompt Leakage

Hidden system instructions get extracted — genuinely dangerous only if they were ever functioning as a real security control.

Key Complications

- Classified by OWASP as non-remediable at the prompt level.
- An exposed prompt is a blueprint of the app's own defenses, not just a leaked secret.
- Inherits LLM01's fundamental unsolvability, since extraction is a form of injection.

Key Mitigations

- Never store secrets in the system prompt
- Design every prompt assuming eventual leakage
- Enforce real security in backend code, not prompt text
- Separate business logic from access control

Anchor Real-World Case

Bing Chat "Sydney" (2023) & GPT Store Extraction Study — Sydney's full system prompt was extracted via social engineering; a later study found a 97.2% extraction success rate across 200+ real custom GPTs.

Case Study Worked Through

"The Loyalty Loophole" — a leaked auto-approval threshold exploited to structure fraudulent redemptions just under the limit.

Hands-On Component

Gandalf, reframed as prompt-extraction practice; multi-product extraction-phrasing test.

LLM08 — Vector and Embedding Weaknesses

RAG-specific: the retrieval pipeline feeding the model — not the model itself — becomes the attack surface, via poisoning, leakage, or inversion.

Key Complications

- The 'trust paradox' — retrieved content is implicitly trusted while user input is carefully validated, though both land in the same prompt.
- Just ~5 documents among millions can achieve 90%+ targeted manipulation success.
- Embedding inversion can reconstruct 50–70% of original source text from 'anonymous' vectors.

Key Mitigations

- Permission-aware vector databases
- Strict tenant isolation
- Validate documents before ingestion
- Defense across ingestion, retrieval, and generation layers

Anchor Real-World Case

Live Enterprise RAG Exfiltration Attack (Jan 2025) — Malicious instructions embedded in a public document caused a live RAG system to leak business intelligence and execute unauthorized API calls.

Case Study Worked Through

"The Five Documents" — a hospital RAG assistant poisoned via five planted memos exploiting an ex-employee's unrevoked upload access.

Hands-On Component

PromptTrace (RAG Poisoning lab) — crafting queries to surface a planted poisoned document.

LLM09 — Misinformation

The model hallucinates confidently, and real harm occurs specifically when a human trusts that output without verification.

Key Complications

- Real legal liability without any intent to deceive — negligent misrepresentation is enough.
- 'Slopsquatting' — attackers publish real malicious packages under names AI tools commonly hallucinate.
- Escalating legal sanctions against professionals who cite unverified AI-generated content.

Key Mitigations

- Ground answers in verified sources (RAG)
- Automatic validation before high-stakes actions
- Mandatory human review for critical outputs
- UX that signals uncertainty rather than false confidence

Anchor Real-World Case

Moffatt v. Air Canada (2024) & Mata v. Avianca (2023) — A chatbot invented a nonexistent refund policy and Air Canada was held liable; separately, lawyers were sanctioned for citing six ChatGPT-hallucinated court cases.

Case Study Worked Through

"The Confident Calculator" — a tax-guidance AI stating a fabricated deduction threshold with total, unhedged confidence.

Hands-On Component

DIY citation-verification exercise across multiple AI products.

LLM10 — Unbounded Consumption

The model's real, metered compute cost is exploited — financially, operationally, or as a path to IP theft.

Key Complications

- Denial of Wallet — cost escalation as the attack goal, with no downtime required.
- Runaway agentic loops can drain budgets with zero malicious actor involved.
- Disproportionately dangerous for smaller AI-native startups on pay-per-token infrastructure.

Key Mitigations

- Per-user/app rate limiting
- Hard budget caps, not just request limits
- Loop detection in agent frameworks
- Real-time cost dashboards and alerting
- Graceful degradation under load

Anchor Real-World Case

Runaway Agent Bill Pattern & MCP Infinite Tool Loops (2025–2026) — Widely documented pattern of open-ended agent instructions ('keep iterating until optimal') consuming large token volumes undetected until the monthly bill arrives.

Case Study Worked Through

Closing topic of the course — reinforced that intent doesn't change the need for hard cost/iteration limits.

Hands-On Component

DIY token-cost estimation and agent iteration-cap comparison exercise.

Hands-On Practice Summary

Each risk was paired with a hands-on component — either a free public platform where one exists, or a self-designed exercise where the risk is primarily architectural rather than directly exploitable in a browser.

Risk	Hands-On Method
LLM01	Gandalf (gandalf.lakera.ai) — extraction-style prompt injection game.
LLM02	DIY session-isolation exercise; OpenAI Redis postmortem mapping.
LLM03	Hugging Face model-card / serialization-format audit; personal OAuth-grant inventory exercise.
LLM04	DIY backdoor-trigger design and red-team-suite thought experiment.
LLM05	PromptTrace labs; live markdown image-tag network request test.
LLM06	PromptTrace (Tool Exploitation lab); personal agent permission-mapping exercise.
LLM07	Gandalf, reframed as prompt-extraction practice; multi-product extraction-phrasing test.
LLM08	PromptTrace (RAG Poisoning lab) — crafting queries to surface a planted poisoned document.
LLM09	DIY citation-verification exercise across multiple AI products.
LLM10	DIY token-cost estimation and agent iteration-cap comparison exercise.

Key Platforms Used

- Gandalf (Lakera) — prompt injection and system-prompt extraction practice, 7 levels plus a self-learning bonus level.
- PromptTrace — RAG poisoning, tool exploitation (LLM06), and defense-bypass labs with full context visibility.
- PortSwigger Web Security Academy — LLM attack labs in a legal sandbox environment.
- Self-designed exercises — used for LLM02, LLM03, LLM04, LLM09, and LLM10, where the risk is primarily an architecture, governance, or verification-habit problem rather than a direct exploit target.

Original Case Studies Completed

Nine original diagnostic case studies were worked through during the week, each requiring identification of the risk category, root cause, and a specific (not generic) mitigation.

Case Study	Risk(s)	Core Scenario
The Silent Summary	LLM01	Indirect prompt injection via a poisoned email read by an AI assistant.
The Helpful Reset	LLM02 / LLM06	A support chatbot triggering account password resets to an attacker-chosen email.
The Forgotten Connection	LLM03	An unrevoked OAuth grant to a third-party AI tool used as a supply-chain entry point.
The Confident Lie	LLM04 / LLM08	A fabricated legal precedent planted into a shared, crowdsourced RAG knowledge base.
The Broken Image	LLM01 / LLM05	Markdown image exfiltration via an auto-rendered AI-drafted support reply.
The Cleanup Job	LLM06	An over-permissioned cloud agent deleting a production backup archive during routine cleanup.
The Loyalty Loophole	LLM07	A leaked auto-approval threshold exploited to structure fraudulent loyalty-point redemptions.
The Five Documents	LLM04 / LLM08	A hospital RAG assistant poisoned via five planted memos through an ex-employee's unrevoked access.
The Confident Calculator	LLM09	A tax-guidance AI stating a fabricated, unhedged deduction figure a user then filed with.

Cross-Cutting Themes Identified

- Timing matters — LLM03/LLM04 compromise happens at build time and is invisible to runtime defenses; LLM01/LLM02/LLM05 happen live, in conversation.
- Most risks don't require an attacker — Air Canada, the Replit agent, and several session-isolation failures caused real harm with zero malicious intent involved.
- Least privilege is the single most recurring control — appearing as a primary or secondary mitigation across LLM01, LLM02, LLM03, LLM06, and LLM08.
- Real incidents compound categories — EchoLeak (LLM01→LLM02), the agent kill-chain (LLM01→LLM06→LLM05), and slopsquatting (LLM09→LLM03) all show risks working together, not in isolation.
- The durable fix usually lives outside the model — architecture, permission boundaries, human checkpoints, and monitoring consistently outperform trying to make the model behave better through instructions alone.

Self-Assessment Against Week 3 Goal

Goal Criterion	Status
Can explain what each of the 10 risks is and why it happens	Complete
Can cite specific, current real-world incidents per risk	Complete
Can name concrete, layered mitigations (not generic advice)	Complete
Has hands-on exposure to at least one exploit/exercise per risk	In progress — see Hands-On Report
Can diagnose a novel scenario and propose a specific fix	Demonstrated across 9 case studies

Next Steps — Week 4 Preview

Per the original roadmap, Week 4 moves into Enterprise AI Security: Responsible AI, AI Governance, Risk Management, Human Review, and Data Classification, using Google's Secure AI Framework (SAIF) and the NIST AI Risk Management Framework as core resources. The goal shifts from "what can go wrong and why" (Week 3) to "how organizations formally govern and manage that risk" — a natural continuation, since nearly every mitigation discussed this week (least privilege, human review gates, provenance tracking, monitoring) is exactly what enterprise AI governance frameworks exist to formalize and enforce at scale.